

Your Automated Execution Does Not Have to be Flaky

Eurostar Webinars Feb 2018

Alan Richardson

- www.eviltester.com/flaky
- www.compendiumdev.co.uk
- [@eviltester](https://twitter.com/eviltester)

Have you experienced flaky test automation? In this webinar, Alan Richardson plans to convince you that you haven't. Instead you have experienced the result of not resolving the causes of Intermittent execution. Alan will explore some common causes and solutions of intermittent behaviour. Why? So you never say the phrase, "flaky tests", ever again.

Flaky Test Automation is Normal

Flaky Test Automation is Normal

Because we have normalized Flaky Test Automation

- "Our Tests are Flaky"
- "Some tests fail Randomly"

"There is nothing so absurd that it has not been said by some philosopher."

Cicero, On Divination, Book II chapter LVIII, section 119 (44 BC)

"Truth happens to an idea. It becomes true, is made true by events."

William James, Lecture VI, Pragmatism's Conception of Truth,
Pragmatism: A New Name for Some Old Ways of Thinking (1907)

How to Normalize Flaky Test Automation

We all know these Test Automation Truths

- "GUI Automation is Flaky"
- "We have to live with 'flakiness'"
- "We shouldn't automate at the GUI"
- "We can only remove flakiness under the GUI"

"Flaky Tests" blames the tests. Not good enough.

It isn't even the "Tests"

- We don't automate Tests
- We automate the execution of steps in a workflow or process
- We automate the execution of a System
- We add condition assertions during that execution

We don't have flaky Tests - we have automated execution that fails. Sometimes on the steps, sometimes on the assertions.

'Flakiness' does not reside at a 'level'

I have seen 'flakiness'

- in Unit Tests
- in API Tests
- in Integration Tests
- in GUI Tests

It is too easy to say 'flaky'

- and then blame 'GUI execution'
- and then blame 'the tool'

Living with flakiness is a choice.

Choose a different approach.

I am not the only person saying this.

- *see references at the end and name drops throughout*
- try to cover something different in this talk

"We designed that flakiness. We are allowing that to happen. We engineered it to be that way. And its our fault that that exists."

Richard Bradshaw, "Your Tests aren't Flaky, You Are!" Selenium Conference 2017

<https://www.youtube.com/watch?v=XnkWkrbzMh0>

Take it more seriously. Describe it differently.

Intermittent

- Occurring at irregular intervals; not continuous or steady.

<https://en.oxforddictionaries.com/definition/intermittent>

Take it more seriously. Describe it differently.

Nondeterministic Algorithm

"a nondeterministic algorithm is an algorithm that, even for the same input, can exhibit different behaviors on different runs"

- https://en.wikipedia.org/wiki/Nondeterministic_algorithm

Flaky is not serious enough.

We do not want to use nondeterministic algorithms for continuous assertions that we are relying on

Your Test Automation is not Flaky

Your automated execution fails intermittently

Don't Blame Tests. Look For Root Causes.

watch Alister Scott's GTAC 2015 talk

I have removed 'flakiness'

- from Unit Tests
- from API Tests
- from Integration Tests
- from GUI Tests

Automated execution does not have to fail intermittently.

How to remove Intermittent Failure from your Automated Execution

1. Care
2. Investigate
3. Do something about it

How to remove Intermittent Failure from your Automated Execution

1. Decide Intermittent Failure is unacceptable
2. Investigate the cause of Intermittent Failure
3. Mitigate
 - Remove the cause
 - actually fix it
 - Implement a retry strategy
 - might obscure bugs
 - Accept Intermittent Results
 - might provide hints at solutions

Take it seriously

We write Automate Assertion checking because we care that those assertions are true for each build of the system.

Determinism is important.

High Level Grouping of Common Causes of Intermittency

- Synchronisation - lack of or poor
- Parallel Execution - interference
- Long Running Tests - too long, too risky
- Automatability - hard to automate system
- Tools - inappropriate or out of date
- State Preconditions - not controlled
- Assertions - wrong or incorrect assumptions
- Data - not controlled

see also Richard Bradshaw and Mark Winteringham "SACRED" Mnemonic.

Will cover Top 3 for each Grouping

Top 3 Common Causes - Synchronisation

- None
- Time Based Synchronisation
- Incorrect App State Synchronisation

Common Solutions - Synchronisation

- Synchronise on States
 - do not rely on framework Synchronisation
- Multiple Intermediate States
 - Consider Latency
- Synchronise in Abstractions not the `@Test` methods
 - unless `@Test` specific

Top 3 Common Causes - Parallel Execution

- Framework not thread safe
- Tests Interfere
- Shared Test Environment

Common Solutions - Parallel Execution

- Independent environments
- Independent Data
- Separate Suites rather than threaded execution
- Create Threadsafe, reusable code
 - Create reusable library abstractions rather than Frameworks
 - Avoid 'static' singleton objects

Top 3 Common Causes - Long Running Tests

- Sequential rather than Model Based
- not delineating between: preconditions, process, assertion
- components tests in flow rather than isolation

Common Solutions - Long Running Tests

- Understand that more actions == more risk
- Synchronise prior to each step
- Consider Model Based Testing
- Create component test and automated execution playgrounds
- Minimum assertions

Top 3 Common Causes - Automatability, Automatizability

Not Testability:

- Application has non-deterministic behaviour
- Hard to Synchronise
- Application fails non-deterministically in live

Common Solutions - Automatability, Automatizability

- Build apps that can be automated
- Non-Deterministic apps need step retry strategies rather than test retry strategies

Top 3 Common Causes - Tools

- Out of Date
- Inappropriate
- Local Tool Infrastructure

Common Solutions - Tools

- Use the right tool for the job
- Keep your tooling environment controlled and up to date
- Change your approach to take latency into account
 - process on server return results
 - return source, process on execution client

Top 3 Common Causes - State Preconditions

- Not Checking State Preconditions at start of test
- Not controlling state preconditions prior to test
- Precondition setup using same tool

Common Solutions - State Preconditions

- control data
- precondition state setup - whatever works
 - http, db, api - 'hack it in'
- avoid dependencies between execution unless a long running test

Top 3 Common Causes - Assumptions Encoded in Assertions

- Assert on an Ordered Set
- Assert on Uncontrolled Data
- Assertion Tolerances

Common Solutions - Assumptions Encoded in Assertions

- Logging so you can interrogate failure afterwards
- Ability to re-run tests with same data and setup

Top 3 Common Causes - Data

- Missing Data
- Externally controlled data
- Uncontrolled Data

Common Solutions - Data

- Create data for each test
- Avoid test dependencies
- Avoid re-using data between tests
- Check data as a precondition
- Data synchronisation on all precondition data

Summary

- Your Test Execution is not 'flaky', it is failing intermittently
- It is possible to remove intermittent failures, even when automating through a GUI
- Commons solutions: synchronisation, data control, environmental isolation

Other talks to watch

- Alister Scott, GTAC 2015: Your Tests Aren't Flaky
 - <https://www.youtube.com/watch?v=hmk1h40shaE>
- Richard Bradshaw, "Your Tests aren't Flaky, You Are!" Selenium Conference 2017
 - <https://www.youtube.com/watch?v=XnkWkrbzMh0>

Other talks to watch

- Craig Schwarzwald, SAY GOODBYE TO THE "F" WORD ... FLAKY NO MORE!
 - https://www.youtube.com/watch?v=2K2M7s_Ups0
- Mark Winteringham - REST APIs and WebDriver: In Perfect Harmony
 - <https://www.youtube.com/watch?v=ugAICZBMOvM>

Search also for: [Flaky Selenium](#), [Flaky Automation](#), [Flaky Test Automation](#)

End

Alan Richardson www.compendiumdev.co.uk

- LinkedIn - [@eviltester](#)
- Twitter - [@eviltester](#)
- Instagram - [@eviltester](#)
- Facebook - [@eviltester](#)
- Youtube - [EvilTesterVideos](#)
- Pinterest - [@eviltester](#)
- Github - [@eviltester](#)
- Slideshare - [@eviltester](#)

BIO

Alan is a Software Development and Testing Coach/Consultant who enjoys testing at a technical level using techniques from psychotherapy and computer science. In his spare time Alan is currently programming a [Twitter client called ChatterScan](#), and [multi-user text adventure game](#). Alan is the author of the books "[Dear Evil Tester](#)", "[Java For Testers](#)" and "[Automating and Testing a REST API](#)". Alan's main website is compendiumdev.co.uk and he blogs at blog.eviltester.com